

חמישה מצבים שנתקעים בהם והדרך החוצה מהם

מטרה

חמישה מצבים שבאמת קורים ב-Git, עם הפקודות המדויקות לצאת מכל אחד, ושורה אחת שמסבירה מה בעצם קרה. שמור את הקובץ הזה פתוח — הוא לא נועד להיקרא מראש, אלא ברגע שנתקעת.

לפני הכול, שתי פקודות שתמיד בטוחות ותמיד עוזרות:

```
git status
git log --oneline -5
```

`git status` אומר לך איפה אתה עומד. `git log --oneline` אומר לך איך הגעת לשם. כמעט תמיד, `git status` כבר מציע לך את הפקודה שאתה צריך. קרא אותו לפני שאתה מנחש.

הכלל שמציל

כמעט שום דבר ב-Git אינו בלתי הפיך אחרי שעשית קומיט. קומיט הוא תמונת מצב מלאה, והיא נשארת. מה שכן אפשר לאבד: שינויים שמעולם לא נכנסו לקומיט. לכן — קומיטים קטנים ותכופים.

1. יש לי שינויים לא שמורים, ו-`git pull` מסרב

מה שתראה:

```
error: Your local changes to the following files would be overwritten by merge:
    week-01/index.html
Please commit your changes or stash them before you merge.
Aborting
```

מה קרה: שינית קובץ, לא עשית קומיט, ובינתיים השתנה אותו קובץ גם בשרת. Git מסרב לדרוס את העבודה שלך — הוא מגן עליך.

הדרך החוצה — אם השינויים שלך חשובים:

```
git add .
git commit -m "wip: my work before pulling"
git pull
```

אם השינויים שלך לא חשובים ואפשר לזרוק אותם:

```
git restore .
git pull
```

אם אינך בטוח — זו האפשרות הבטוחה. היא שמה את השינויים בצד ומחזירה אותם אחר כך:

```
git stash
git pull
git stash pop
```

. `git restore` מוחק את השינויים שלך ואי אפשר לשחזר אותם. אם יש ספק – `git stash`.

2. קונפליקט מיזוג

מה שתראה:

```
Auto-merging week-01/index.html
CONFLICT (content): Merge conflict in week-01/index.html
Automatic merge failed; fix conflicts and then commit the result.
```

ובתוך הקובץ:

```
<<<<<< HEAD
<h1>My personal site</h1>
=====
<h1>Maya Cohen</h1>
>>>>>> 3f2a91c
```

מה קרה: אותה שורה שונתה בשני מקומות, ו-Git לא יכול להחליט בשבילך. **זה לא נזק.**

הדרך החוצה:

1. פתח את הקובץ ב-VS Code. הוא יסמן את האזור וייתן לך כפתורים: `Accept Current Change` (שלך) · `Accept Incoming Change` (של השרת) · `Accept Both Changes`.
2. **מחק את כל שלוש שורות הסימון** – `<<<<<<` , `=====` ו- `>>>>>>` . אם השארת אותן, הן ייכנסו לקומיט וההגשה שלך תיפול בוולידציה.
3. השאר את הגרסה שאתה רוצה, ואז:

```
git add week-01/index.html
git commit -m "fix: resolve merge conflict in week 1 page"
git push
```

אם התבלבלת לגמרי ואתה רוצה להתחיל את המיזוג מחדש:

```
git merge --abort
```

זה מחזיר אותך בדיוק למצב שלפני `git pull`.

3. עשיתי קומיט על משהו לא נכון

שכחתי קובץ, או שההודעה שגויה – והקומיט עדיין לא נדחף

```
git add week-01/images/portrait.svg
git commit --amend -m "feat: add about section with portrait"
```

`--amend` מחליף את הקומיט האחרון בקומיט חדש.

רוצה לבטל את הקומיט האחרון ולהחזיר את השינויים לעריכה

```
git reset --soft HEAD~1
```

הקומיט נעלם, הקבצים שלך נשארים בדיוק כפי שהם. זו האפשרות הבטוחה, וכמעט תמיד זו הנכונה.

קומיטתי בטעות `node_modules` או קובץ ענק

```
git rm -r --cached node_modules
echo "node_modules/" >> .gitignore
git add .gitignore
git commit -m "chore: stop tracking node_modules"
```

`--cached` מסיר את הקובץ מהמעקב של Git ומשאיר אותו על הדיסק. בלי `--cached` הוא נמחק.

הקומיט כבר נדחף

אל תשתמש ב- `--amend` ואל תעשה `reset` לקומיט שכבר בשרת. זה משנה היסטוריה שכבר פורסמה. במקום זה, בטל אותו בקומיט חדש:

```
git revert HEAD
git push
```

`revert` יוצר קומיט חדש שמבטל את השינוי. ההיסטוריה נשארת שלמה וכנה – וזה בדיוק מה שאנחנו רוצים לראות.

שים לב

אל תשתמש לעולם ב- `git push --force` במאגר של הקורס. הוא יכול למחוק קומיטים שהוגשו, וה-SHA שהגשת במודל יפסיק להתקיים. הגשה שה-SHA שלה לא נמצא במאגר נבדקת לפי הקומיט האחרון שקדם למועד.

4. HEAD מנותק (detached HEAD)

מה שתראה:

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

מה קרה: עשית `git checkout` לקומיט מסוים במקום לענף. אתה מסתכל על תמונת מצב ישנה. **זה לא נזק ואף אחד לא איבד כלום** – אבל קומיט שתעשה כאן לא שייך לשום ענף, ויאבד.

הדרך החוצה – אם לא שינית כלום:

```
git switch main
```

אם כן עשית כאן קומיט ואתה רוצה לשמור אותו:

```
git branch rescue-my-work
git switch main
git merge rescue-my-work
```

השורה הראשונה מצמידה שם לקומיט שאחרת היה נעלם. עשה אותה **לפני** שאתה עוזב.

5. דחפתי לענף הלא נכון

מה קרה: עבדת על ענף אחר – או יצרת ענף בטעות – והעבודה לא נמצאת ב- `main`.

קודם כול, ראה איפה אתה ומה קיים:

```
git branch -a
git status
```

להעביר את הקומיטים ל- `main`:

```
git switch main
git merge my-wrong-branch
git push
```

אם עדיין לא עשית קומיט, ופשוט אתה על הענף הלא נכון:

```
git switch main
```

השינויים שלא בקומיט עוברים איתך.

מה שמגישים

בקורס הזה מגישים תמיד מ- main . עבודה שיושבת בענף אחר ולא מוזגה לא נבדקת, גם אם היא נדחפה. לפני כל הגשה ודא ששתי הפקודות האלה מסכימות:

```
git branch --show-current # צריך להחזיר main
git log -1 --format=%H     # שאתה מגיש SHA-זה ה
```

הטעויות שחוזרות בכל שנה

הטעות	מה קורה	מה לעשות במקום
להעלות קבצים דרך ממשק הדפדפן של GitHub	ההיסטוריה המקומית והשרת מתפצלים, וכל pull הופך לקונפליקט	רק git add / git commit / git push
קומיט ענק אחד בסוף	אי אפשר לראות מה השתנה, ואי אפשר לחזור צעד אחד אחורה	קומיט אחרי כל שלב שהושלם
. git add בלי להסתכל	node_modules , DS_Store . וקבצים זמניים נכנסים למאגר	git status לפני, תמיד
לשכוח git pull לפני שמתחילים	ממשיכים מגרסה ישנה ומייצרים קונפליקט מיותר	git pull כשפותחים את המחשב
הודעות כמו update או fix	אף אחד — כולל אתה בשבוע 13 — לא יידע מה קרה שם	feat: / fix: / docs: ומשפט אמיתי
git push --force כדי "לסדר"	קומיטים שהוגשו נמחקים, וה-SHA שהגשת מפסיק להתקיים	git revert , תמיד

תחיליות הקומיט שאנחנו משתמשים בהן: feat: פיצ'ר חדש · fix: תיקון באג · style: עיצוב וסידור בלבד · refactor: שינוי מבנה בלי שינוי התנהגות · docs: תיעוד · chore: תחזוקה.

ואם שום דבר כאן לא מתאים

אל תמחק את התיקיה ותתחיל מחדש. קודם שמור עותק:

```
cd ..
cp -r frontend-2027-123456789 frontend-2027-backup
```

ואז פנה אליי עם הפלט המלא של git status ושל git log --oneline -5 . עם שתי אלה אפשר לשחזר כמעט כל מצב.